Naive Bayes Text Classification

1. Import Required Libraries

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

Purpose

CountVectorizer

Converts text into a matrix of word counts.

Example:

```
"free lottery prize"
```

becomes numerical features such as:

```
[1, 1, 1]
```

for words:

```
free, lottery, prize
```

MultinomialNB

Implements the **Multinomial Naive Bayes Algorithm**, which is widely used for:

- Spam detection
- Email filtering
- Sentiment analysis
- Document classification

2. Training Dataset

```
documents = [
    "Congratulations you won a lottery",
    "Claim your free prize now",
    "Win money instantly",
    "Meeting scheduled for tomorrow",
    "Please submit the assignment",
    "Project discussion at 10 AM"
```

```
]
```

These six training messages are examples used to teach the classifier.

Spam Messages

```
Congratulations you won a lottery
Claim your free prize now
Win money instantly
```

Ham Messages

```
Meeting scheduled for tomorrow
Please submit the assignment
Project discussion at 10 AM
```

3. Define Labels

```
labels = [
    "Spam",
    "Spam",
    "Spam",
    "Ham",
    "Ham",
    "Ham"
]
```

Each document is assigned a category.

Message	Label
Congratulations you won a lottery	Spam
Claim your free prize now	Spam
Win money instantly	Spam
Meeting scheduled for tomorrow	Ham
Please submit the assignment	Ham
Project discussion at 10 AM	Ham

4. Create Count Vectorizer

```
vectorizer = CountVectorizer()
```

Purpose

Machine learning algorithms cannot work directly on text.

The vectorizer converts words into numbers.

Example vocabulary:

```
assignment
discussion
free
lottery
meeting
money
prize
project
submit
won
```

Each word becomes a feature.

5. Convert Text into Feature Matrix

```
X = vectorizer.fit_transform(documents)
```

What Happens?

The vectorizer:

1. Learns the vocabulary.
2. Counts word occurrences.
3. Creates a document-term matrix.

Example:

Document	free	lottery	meeting
free lottery prize	1	1	0
meeting tomorrow	0	0	1

This numerical representation is used for training.

6. Create Naive Bayes Classifier

```
model = MultinomialNB()
```

Why Multinomial Naive Bayes?

It works particularly well when:

- Features are word counts.
- Text classification is required.
- Vocabulary size is large.

It is one of the most popular algorithms for spam filtering.

7. Train the Model

```
model.fit(X, labels)
```

What Happens?

The classifier learns:

Spam-Related Words

```
won
lottery
free
prize
money
instantly
claim
```

Ham-Related Words

```
meeting
project
discussion
assignment
submit
tomorrow
```

The model computes probabilities for each class based on word occurrences.

8. Test Message

```
test_message = ["free lottery prize"]
```

This is a new message that was not part of training data.

9. Convert Test Message

```
test_vector = vectorizer.transform(test_message)
```

The same vocabulary learned during training is applied to the test message.

Example representation:

```
free    → 1
lottery → 1
prize   → 1
```

The message becomes a numerical vector.

10. Predict the Category

```
prediction = model.predict(test_vector)
```

The Naive Bayes classifier calculates:

and

using Bayes' Theorem:

where:

- (C) = Class (Spam or Ham)
- (X) = Message

The class with the higher probability is selected.

11. Display Output

```
print("Message:", test_message[0])
print("Predicted Class:", prediction[0])
```

Expected Output:

```
Message: free lottery prize
Predicted Class: Spam
```

because all three words are strongly associated with spam messages in the training dataset.

How Naive Bayes Works Here

For the message:

```
free lottery prize
```

The model observes:

Word	Seen in Spam?	Seen in Ham?
free	Yes	No
lottery	Yes	No
prize	Yes	No

Therefore:

```
P(Spam) > P(Ham)
```

Prediction:

```
Spam
```

Why Is It Called "Naive"?

Naive Bayes assumes that all features (words) are independent:

is approximated as

This assumption is not always true in real language, but the algorithm often works surprisingly well. 📌

Real-World Applications

Naive Bayes is used for:

- Email Spam Detection
- SMS Filtering
- Sentiment Analysis
- News Categorization
- Customer Feedback Analysis
- Chat Message Classification
- Document Organization

Program Workflow

```
Training Messages
        ↓
Count Vectorizer
        ↓
Document-Term Matrix
        ↓
Train Naive Bayes Model
        ↓
New Message
        ↓
 Vectorization
        ↓
 Probability Calculation
        ↓
 Spam / Ham Prediction
```

Key Concepts Used

Concept	Purpose
CountVectorizer	Converts text into word-count features
Bag of Words	Represents documents numerically
Naive Bayes	Probabilistic classifier
Supervised Learning	Learns from labeled examples
Text Classification	Assigns documents to categories

Interview Questions

Q1. What is CountVectorizer?

Answer: CountVectorizer converts text into a matrix of token counts, allowing machine learning algorithms to process textual data.

Q2. Why is Naive Bayes suitable for spam detection?

Answer: It is fast, simple, works well with word frequencies, and performs effectively on text classification problems.

Q3. What will be the output of this program?

Answer:

```
Message: free lottery prize
Predicted Class: Spam
```

because the words *free*, *lottery*, and *prize* are strongly associated with the Spam class.

Time Complexity

Let:

- **N** = Number of training documents
- **V** = Vocabulary size

Training

```
CountVectorizer : O(N × V)
Naive Bayes Training : O(N × V)
```

Prediction

```
O(V)
```

Overall, Naive Bayes is considered a very efficient algorithm for text classification tasks.

Key Takeaway

This program builds a simple **spam detection system**. It converts text into numeric word-count features using **CountVectorizer**, trains a **Multinomial Naive Bayes** classifier on labeled examples, and predicts whether a new message belongs to the **Spam** or **Ham** category.